

Index

Miao Qiao

The University of Auckland



Outline

- Basic Concepts
- Ordered Indices
- B⁺-Tree Index Files
- B-Tree Index Files
- Hashing
- Write-optimized indices

Hashing

Example of Hash Index

bucket 0

76766	

bucket 1

45565	
76543	

bucket 2

22222	

bucket 3

10101	

bucket 4

bucket 5

15151	
33456	

58583	
98345	

bucket 6

83821	

bucket 7

12121	
32343	

76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
33465	Gold	Physics	87000

hash index on *instructor*, on attribute *ID*

Static Hashing

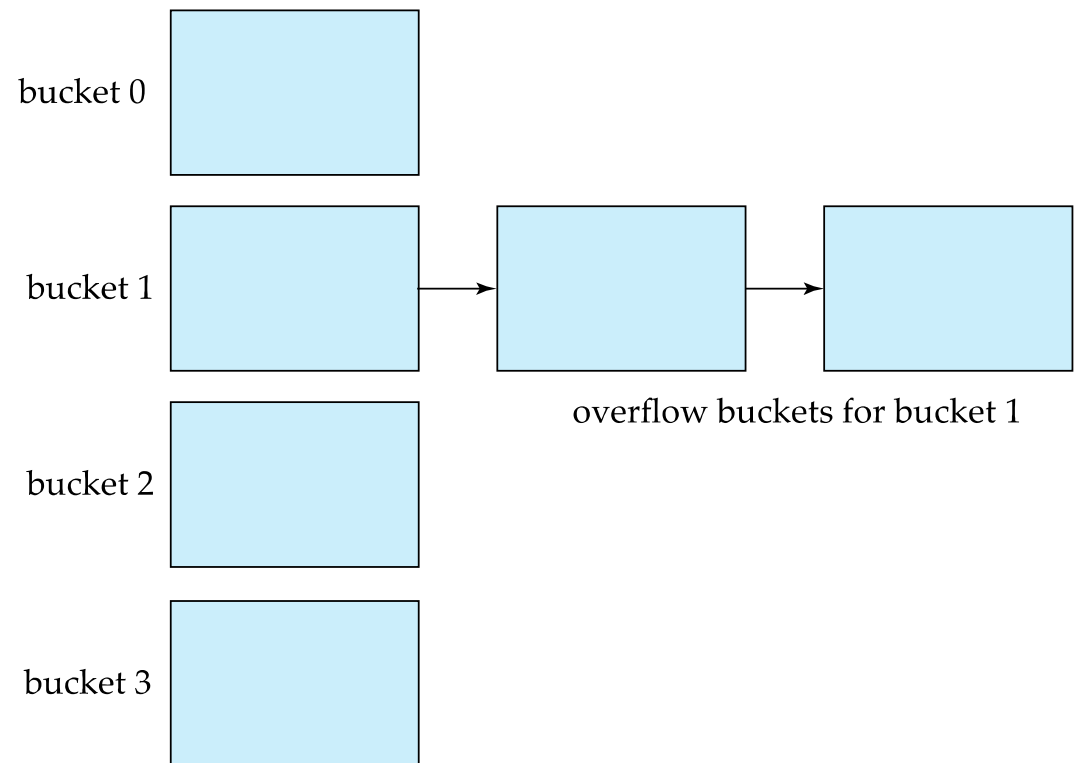
- A **bucket** is a unit of storage containing one or more entries (a bucket is typically a disk block).
 - we obtain the bucket of an entry from its search-key value using a **hash function**
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B .
- Hash function is used to locate entries for access, insertion as well as deletion.
- Entries with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate an entry.
- In a **hash index**, buckets store entries with pointers to records
- In a **hash file-organization** buckets store records

Handling of Bucket Overflows

- Bucket overflow can occur because of
 - Insufficient buckets
 - Skew in distribution of records. This can occur due to two reasons:
 - multiple records have same search-key value
 - chosen hash function produces non-uniform distribution of key values
- Although the probability of bucket overflow can be reduced, it cannot be eliminated; it is handled by using ***overflow buckets***.

Handling of Bucket Overflows (Cont.)

- **Overflow chaining** – the overflow buckets of a given bucket are chained together in a linked list.
- Above scheme is called **closed addressing** (also called **closed hashing** or **open hashing** depending on the book you use)
 - An alternative, called **open addressing** (also called **open hashing** or **closed hashing** depending on the book you use) which does not use overflow buckets, is not suitable for database applications.



Example of Hash File Organization

Hash file organization of *instructor* file, using *dept_name* as key.

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

Deficiencies of Static Hashing

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time.
 - If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows.
 - If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
 - If database shrinks, again space will be wasted.
- One solution: periodic re-organization of the file with a new hash function
 - Expensive, disrupts normal operations
- Better solution: allow the number of buckets to be modified dynamically.

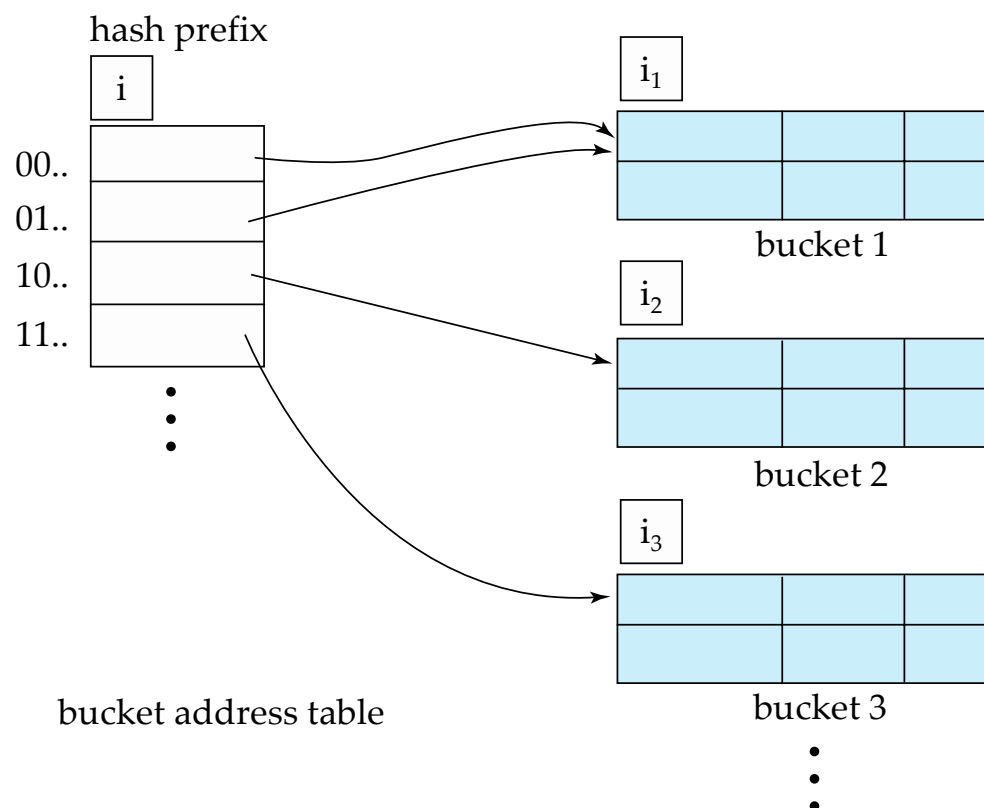
Dynamic Hashing

- Periodic rehashing
 - If number of entries in a hash table becomes (say) 1.5 times size of hash table,
 - create new hash table of size (say) 2 times the size of the previous hash table
 - Rehash all entries to new table
- Linear Hashing
 - Do rehashing in an incremental manner
- Extendable Hashing
 - Tailored to disk based hashing, with buckets shared by multiple hash values
 - Doubling of # of entries in hash table, without doubling # of buckets

Extendable Hashing

- **Extendable hashing** – one form of dynamic hashing
 - Hash function generates values over a large range — typically b -bit integers, with $b = 32$.
 - At any time use only a prefix of the hash function to index into a table of bucket addresses.
 - Let the length of the prefix be i bits, $0 \leq i \leq 32$.
 - Bucket address table size = 2^i . Initially $i = 0$
 - Value of i grows and shrinks as the size of the database grows and shrinks.
 - Multiple entries in the bucket address table may point to a bucket (why?)
 - Thus, actual number of buckets is $< 2^i$
 - The number of buckets also changes dynamically due to coalescing and splitting of buckets.

General Extendable Hash Structure



In this structure, $i_2 = i_3 = i$, whereas $i_1 = i - 1$ (see next slide for details)

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
 - All the entries that point to the same bucket have the same values on the first i_j bits.
- To locate the bucket containing search-key K_j :
 1. Compute $h(K_j) = X$
 2. Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value K_j
 - follow same procedure as look-up and locate the bucket, say j .
 - If there is room in the bucket j insert record in the bucket.
 - Else the bucket must be split and insertion re-attempted (next slide.)
 - Overflow buckets used instead in some cases (will see shortly)

Insertion in Extendable Hash Structure (Cont.)

To split a bucket j when inserting record with search-key value K_j :

- If $i > i_j$ (more than one pointer to bucket j)
 - allocate a new bucket z , and set $i_j = i_z = (i_j + 1)$
 - Update the second half of the bucket address table entries originally pointing to j , to point to z
 - remove each record in bucket j and reinsert (in j or z)
 - recompute new bucket for K_j and insert record in the bucket (further splitting is required if the bucket is still full)
- If $i = i_j$ (only one pointer to bucket j)
 - If i reaches some limit b , or too many splits have happened in this insertion, create an overflow bucket
 - Else
 - increment i and double the size of the bucket address table.
 - replace each entry in the table by two entries that point to the same bucket.
 - recompute new bucket address table entry for K_j
Now $i > i_j$ so use the first case above.

Deletion in Extendable Hash Structure

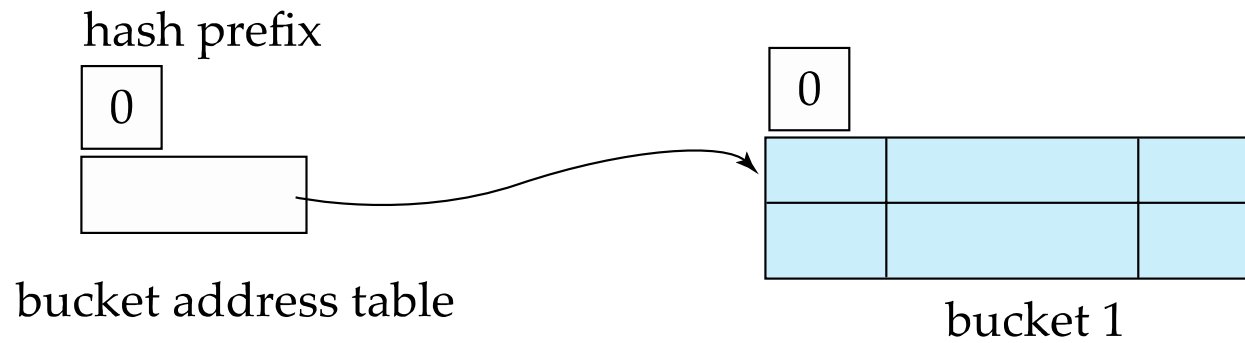
- To delete a key value,
 - locate it in its bucket and remove it.
 - The bucket itself can be removed if it becomes empty (with appropriate updates to the bucket address table).
 - Coalescing of buckets can be done (can coalesce only with a “*buddy*” bucket having same value of i_j and same $i_j - 1$ prefix, if it is present)
 - Decreasing bucket address table size is also possible
 - Note: decreasing bucket address table size is an expensive operation and should be done only if number of buckets becomes much smaller than the size of the table

Use of Extendable Hash Structure: Example

<i>dept_name</i>	$h(\text{dept_name})$
Biology	0010 1101 1111 1011 0010 1100 0011 0000
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 1101
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 1111
Finance	1010 0011 1010 0000 1100 0110 1001 1111
History	1100 0111 1110 1101 1011 1111 0011 1010
Music	0011 0101 1010 0110 1100 1001 1110 1011
Physics	1001 1000 0011 1111 1001 1100 0000 0001

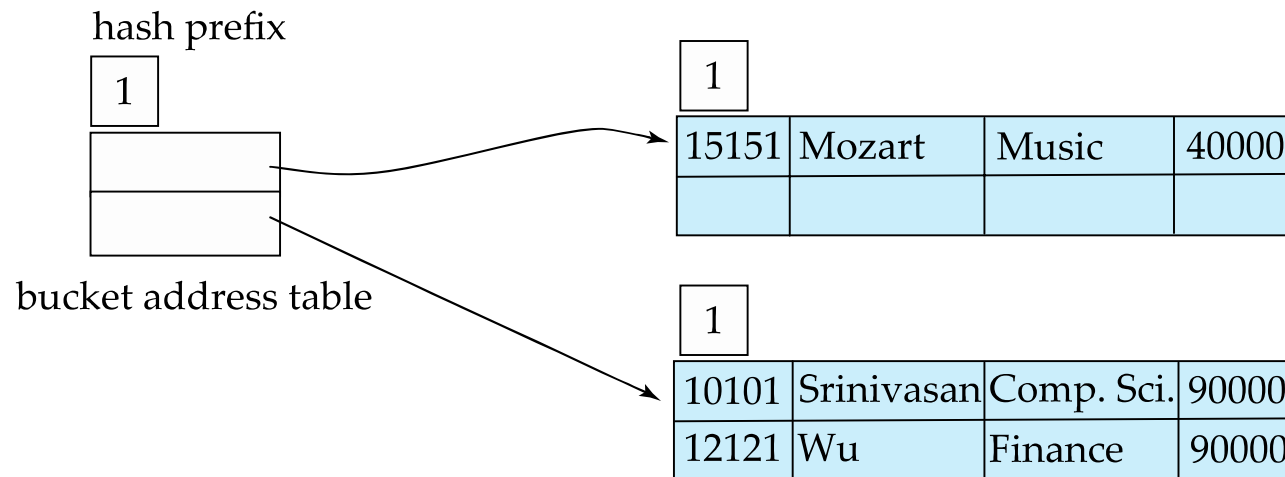
Example (Cont.)

- Initial hash structure; bucket size = 2



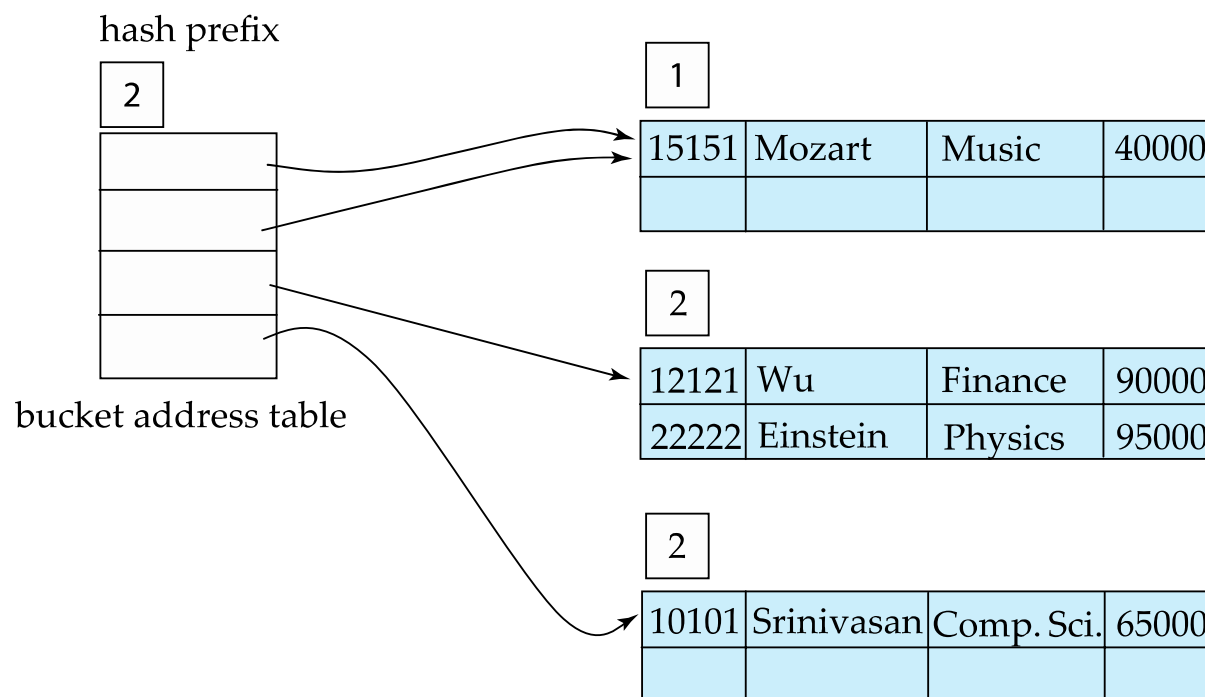
Example (Cont.)

- Hash structure after insertion of “Mozart”, “Srinivasan”, and “Wu” records



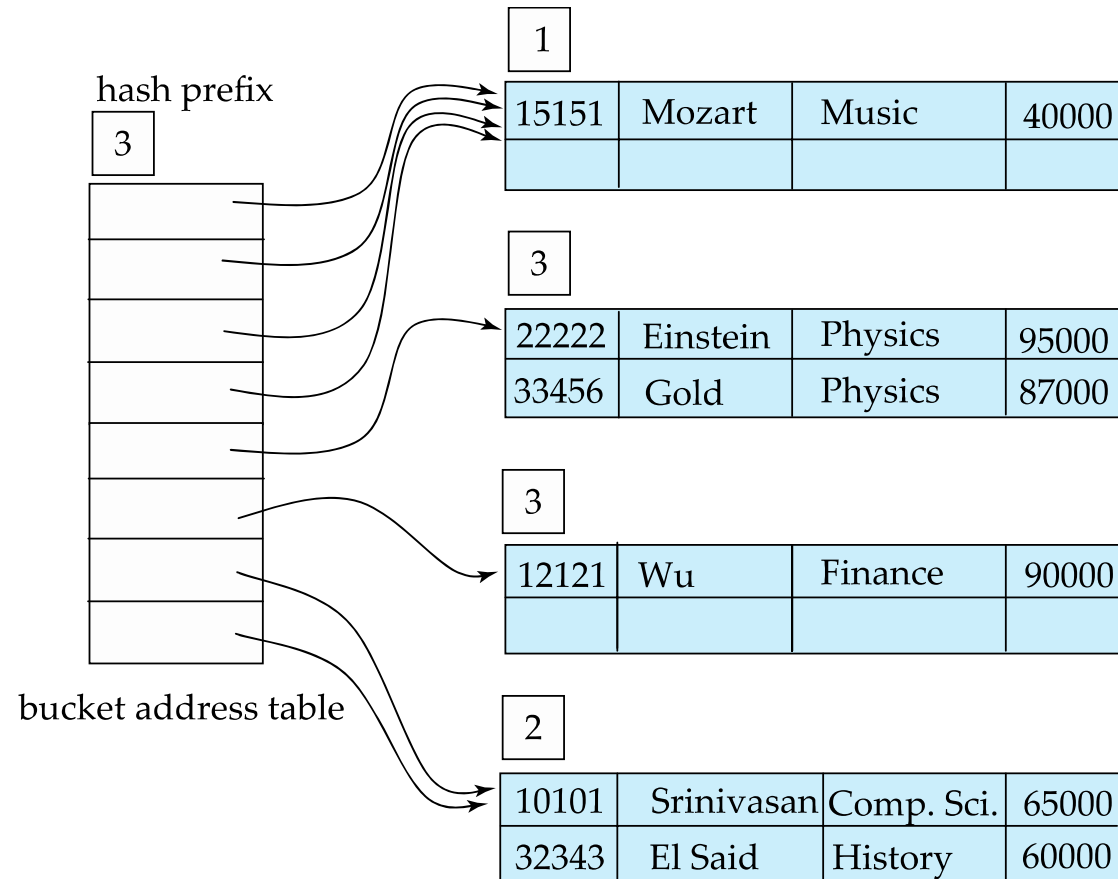
Example (Cont.)

- Hash structure after insertion of Einstein record



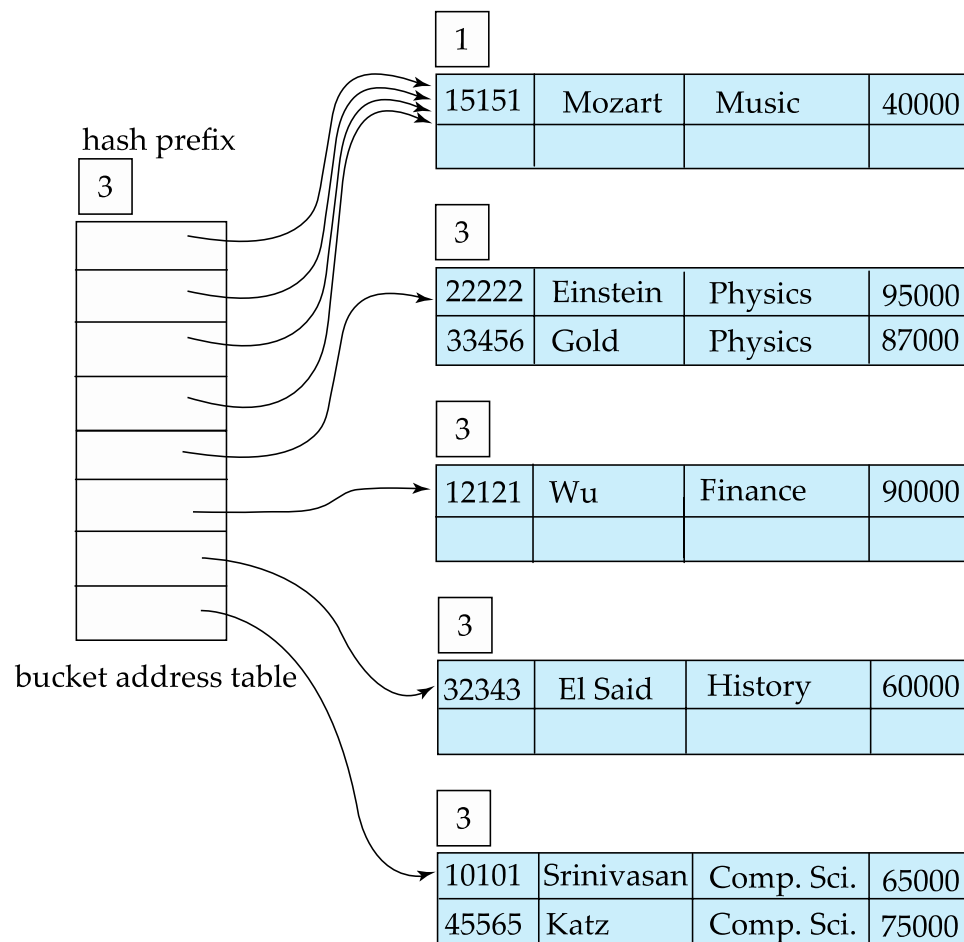
Example (Cont.)

- Hash structure after insertion of Gold and El Said records

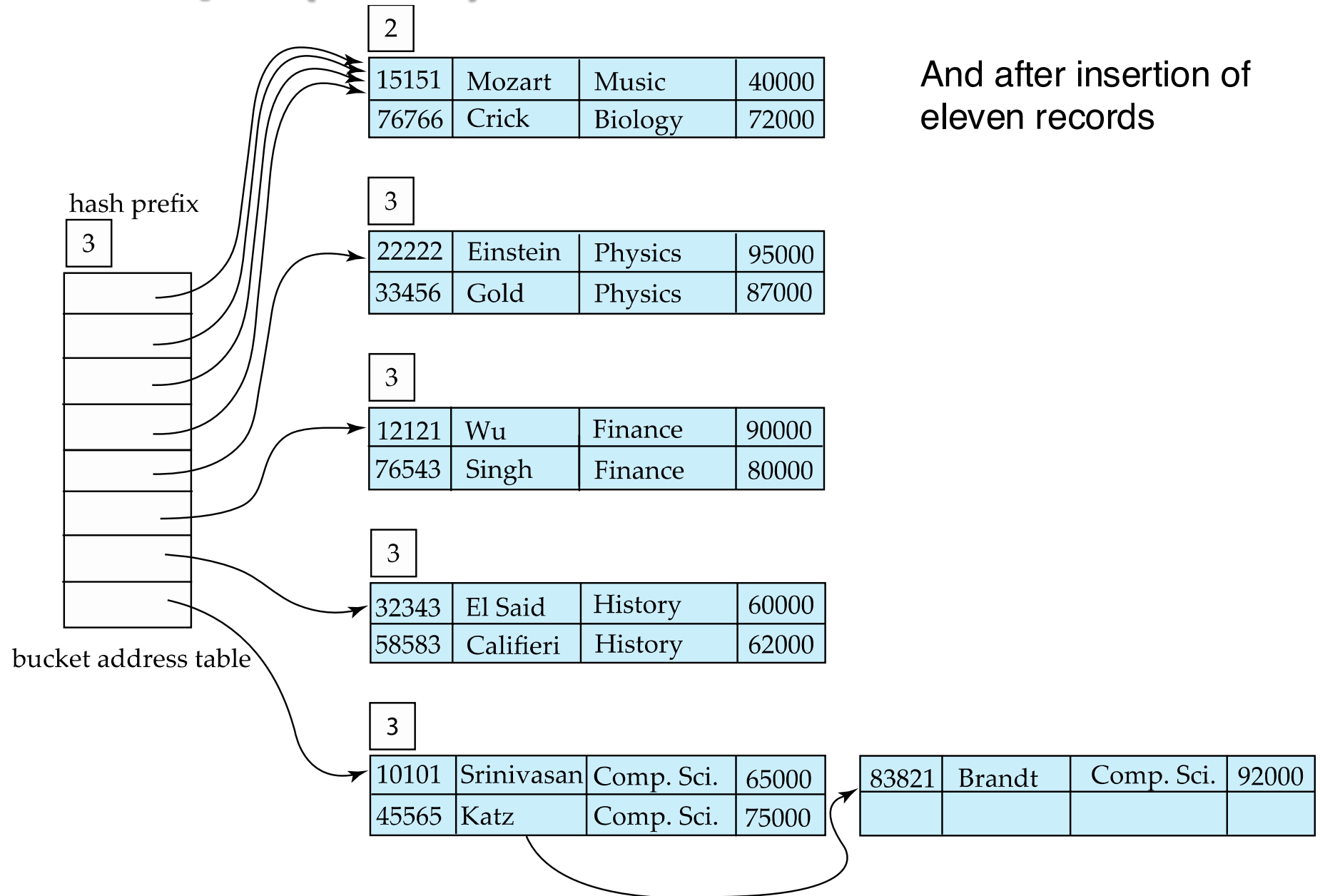


Example (Cont.)

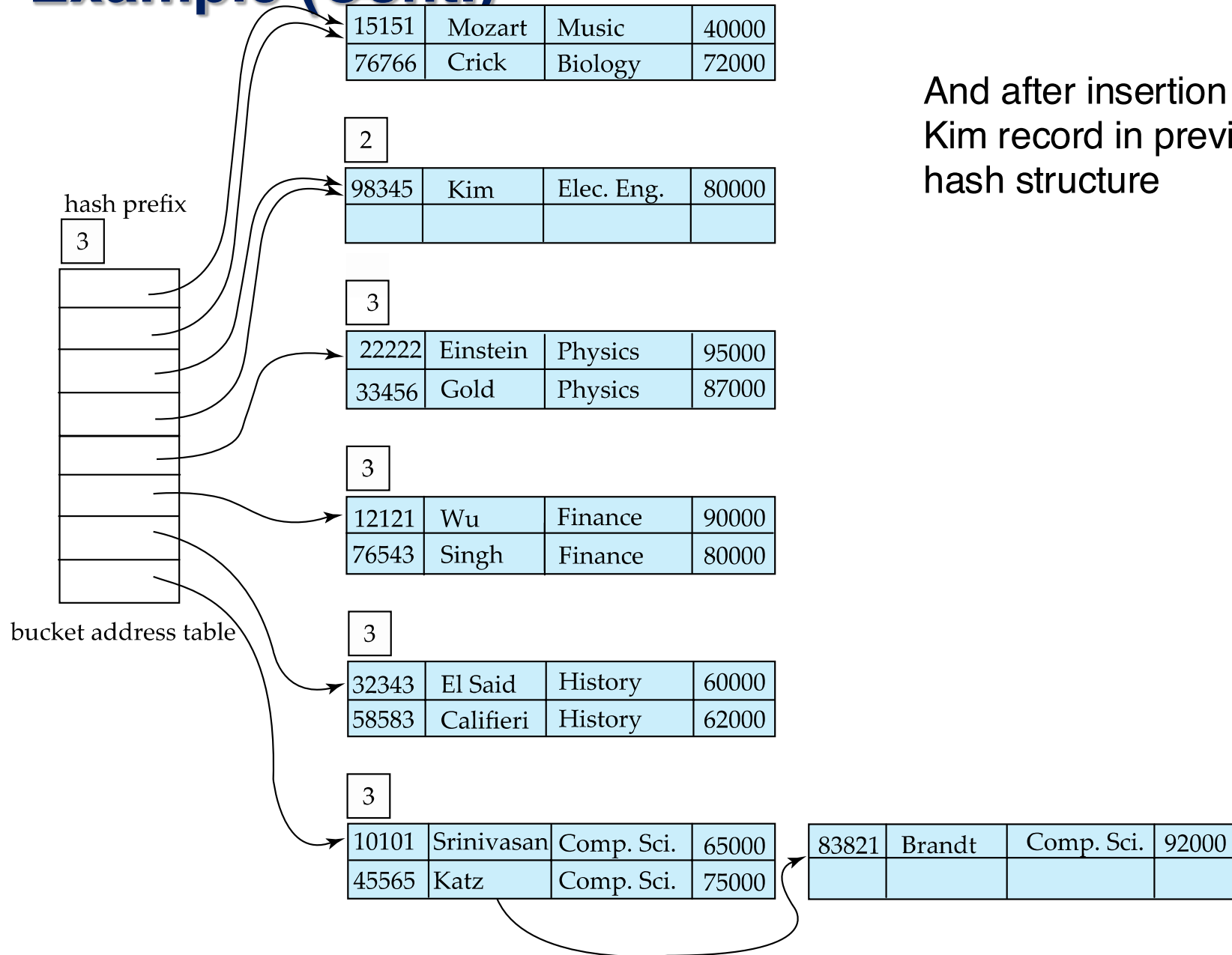
- Hash structure after insertion of Katz record



Example (Cont.)



Example (Cont.)



Extendable Hashing vs. Other Schemes

- Benefits of extendable hashing:
 - Hash performance does not degrade with growth of file
 - Minimal space overhead
- Disadvantages of extendable hashing
 - Extra level of indirection to find desired record
 - Bucket address table may itself become very big (larger than memory)
 - Cannot allocate very large contiguous areas on disk either
 - Solution: B⁺-tree structure to locate desired record in bucket address table
 - Changing size of bucket address table is an expensive operation
- **Linear hashing** is an alternative mechanism
 - Allows incremental growth of its directory (equivalent to bucket address table)
 - At the cost of more bucket overflows

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
 - Hashing is generally better at retrieving records having a specified value of the key.
 - If range queries are common, ordered indices are to be preferred
- In practice:
 - PostgreSQL supports hash indices, but discourages use due to poor performance
 - Oracle supports static hash organization, but not hash indices
 - SQLServer supports only B⁺-trees

Multiple-Key Access

- Use multiple indices for certain types of queries.
- Example:
select *ID*
from *instructor*
where *dept_name* = “Finance” **and** *salary* = 80000
- Possible strategies for processing query using indices on single attributes:
 1. Use index on *dept_name* to find instructors with department name Finance; test *salary* = 80000
 2. Use index on *salary* to find instructors with a salary of \$80000; test *dept_name* = “Finance”.
 3. Use *dept_name* index to find pointers to all records pertaining to the “Finance” department. Similarly use index on *salary*. Take intersection of both sets of pointers obtained.

Indices on Multiple Keys

- **Composite search keys** are search keys containing more than one attribute
 - E.g., (*dept_name*, *salary*)
- Lexicographic ordering: $(a_1, a_2) < (b_1, b_2)$ if either
 - $a_1 < b_1$, or
 - $a_1 = b_1$ and $a_2 < b_2$

Indices on Multiple Attributes

Suppose we have an index on combined search-key
(*dept_name*, *salary*).

- With the **where** clause
 where *dept_name* = “Finance” **and** *salary* = 80000
the index on (*dept_name*, *salary*) can be used to fetch only records that satisfy both conditions.
 - Using separate indices is less efficient — we may fetch many records (or pointers) that satisfy only one of the conditions.
- Can also efficiently handle
 where *dept_name* = “Finance” **and** *salary* < 80000
- But cannot efficiently handle
 where *dept_name* < “Finance” **and** *balance* = 80000
 - May fetch many records that satisfy the first but not the second condition

Other Features

- **Covering indices**
 - Add extra attributes to index so (some) queries can avoid fetching the actual records
 - Store extra attributes only at leaf
 - Why?
- Particularly useful for secondary indices
 - Why?

Creation of Indices

- Example
 - create index** *takes_pk* **on** *takes* (*ID*, *course_ID*, *year*, *semester*, *section*)
 - drop index** *takes_pk*
- Most database systems allow specification of type of index, and clustering.
- Indices on primary key created automatically by all databases
 - Why?
- Some database also create indices on foreign key attributes
 - Why might such an index be useful for this query:
 - *takes* ⋈ $\sigma_{name='Shankar'}(student)$
- Indices can greatly speed up lookups, but impose cost on updates
 - Index tuning assistants/wizards supported on several databases to help choose indices, based on query and update workload

Index Definition in SQL

- Create an index

create index <index-name> **on** <relation-name>
(<attribute-list>)

E.g.,: **create index** *b-index* **on** *branch(branch_name)*

- Use **create unique index** to indirectly specify and enforce the condition that the search key is a candidate key is a candidate key.
 - Not really required if SQL **unique** integrity constraint is supported
- To drop an index

drop index <index-name>

- Most database systems allow specification of type of index, and clustering.

Write Optimized Indices

- Performance of B⁺-trees can be poor for write-intensive workloads
 - One I/O per leaf, assuming all internal nodes are in memory
 - With magnetic disks, < 100 inserts per second per disk
 - With flash memory, one page overwrite per insert
- Two approaches to reducing cost of writes
 - **Log-structured merge tree**
 - **Buffer tree**

Bloom Filters

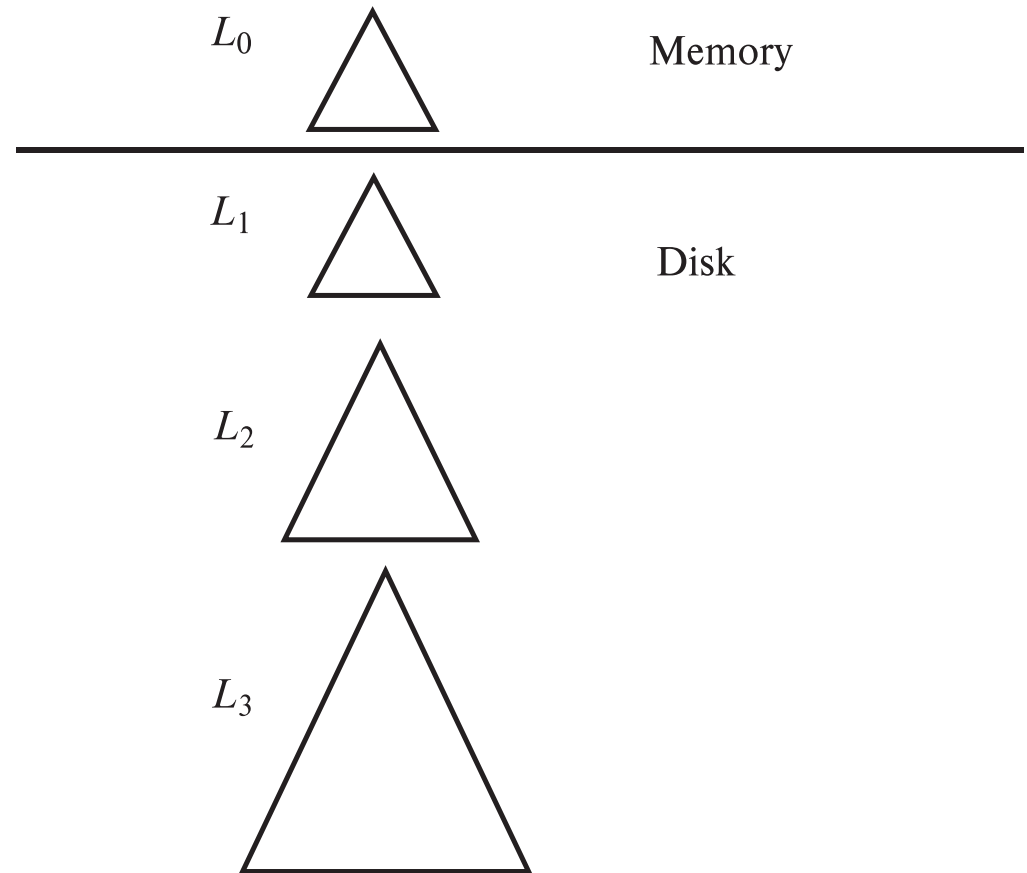
- A **bloom filter** is a probabilistic data structure used to check membership of a value in a set
 - May return true (with low probability) even if an element is not present
 - But never returns false if an element is present
 - Used to filter out irrelevant sets
- Key data structure is a single bitmap
 - For a set with n elements, typical bitmap size is $10n$
- Uses multiple independent hash functions
- With a single hash function $h()$ with range=number of bits in bitmap:
 - For each element s in set S compute $h(s)$ and set bit $h(s)$
 - To query an element v compute $h(v)$, and check if bit $h(v)$ is set
- Problem with single hash function: significant chance of false positive due to hash collision
 - 10% chance with $10n$ bits

Bloom Filters (Cont.)

- Key idea of Bloom filter: reduce false positives by use multiple hash functions $h_i()$ for $i = 1..k$
 - For each element s in set S for each i compute $h_i(s)$ and set bit $h_i(s)$
 - To query an element v for each i compute $h_i(v)$, and check if bit $h_i(v)$ is set
 - If bit $h_i(v)$ is set for every i then report v as present in set
 - Else report v as absent
 - With $10n$ bits, and $k = 7$, false positive rate reduces to 1% instead of 10% with $k = 1$

Log Structured Merge (LSM) Tree

- Consider only inserts/queries for now
- Records inserted first into in-memory tree (L_0 tree)
- When in-memory tree is full, records moved to disk (L_1 tree)
 - B^+ -tree constructed using bottom-up build by merging existing L_1 tree with records from L_0 tree
- When L_1 tree exceeds some threshold, merge into L_2 tree
 - And so on for more levels
 - Size threshold for L_{i+1} tree is k times size threshold for L_i tree



LSM Trees (Cont.)

- Deletion handled by adding special “delete” entries
 - Lookups will find both original entry and the delete entry, and must return only those entries that do not have matching delete entry
 - When trees are merged, if we find a delete entry matching an original entry, both are dropped.
- Update handled using insert+delete
- LSM trees were introduced for disk-based indices
 - But useful to minimize erases with flash-based indices
 - The stepped-merge variant of LSM trees is used in many BigData storage systems
 - Google BigTable, Apache Cassandra, MongoDB
 - And more recently in SQLite4, LevelDB, and MyRocks storage engine of MySQL

LSM Tree (Cont.)

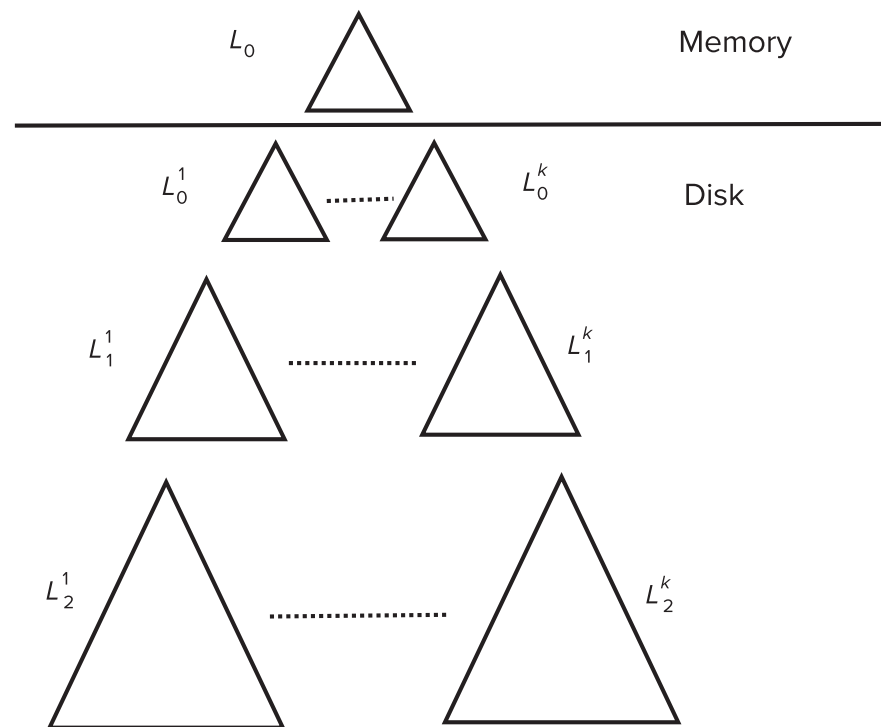
- Benefits of LSM approach
 - Inserts are done using only sequential I/O operations
 - Leaves are full, avoiding space wastage
 - Reduced number of I/O operations per record inserted as compared to normal B⁺-tree (up to some size)
 - If each leaf has m entries, m/k entries merged in using 1 IO
 - Total I/O operations: $k/m \log_k(I/M)$ where I = total number of entries, and M is the size of L_0 tree.
- Drawback of LSM approach
 - Queries have to search multiple trees
 - Entire content of each level copied multiple times

Optimizations of LSM

- **Rolling merge**
- LSM/Stepped Merge often implemented on a partitioned relation
 - Each partition size set to some max, split if over-sized
 - Spread partitions over multiple machines

Stepped Merge Index

- Stepped-merge index: variant of LSM tree with k trees at each level on disk
 - When all k indices exist at a level, merge them into one index of next level.
 - Reduces write cost compared to LSM tree
- But queries are even more expensive since many trees need to be queried
- Optimization for point lookups
 - Compute Bloom filter for each tree and store in-memory
 - Query a tree only if Bloom filter returns a positive result



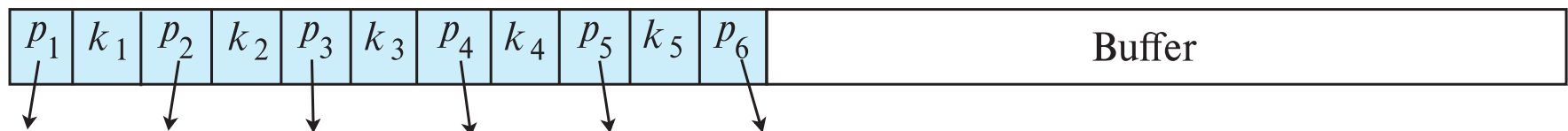
LSM Trees (Cont.)

- Deletion handled by adding special “delete” entries
 - Lookups will find both original entry and the delete entry, and must return only those entries that do not have matching delete entry
 - When trees are merged, if we find a delete entry matching an original entry, both are dropped.
- Update handled using insert + delete
- LSM trees were introduced for disk-based indices
 - But useful to minimize erases with flash-based indices
 - The stepped-merge variant of LSM trees is used in many BigData storage systems
 - Google BigTable, Apache Cassandra, MongoDB
 - And more recently in SQLite4, LevelDB, and MyRocks storage engine of MySQL

Buffer Tree

- Alternative to LSM tree
- Key idea: each internal node of B⁺-tree has a buffer to store inserts
 - Inserts are moved to lower levels when buffer is full
 - With a large buffer, many records are moved to lower level each time
 - Per record I/O decreases correspondingly
- Benefits
 - Less overhead on queries
 - Can be used with any tree index structure
 - Used in PostgreSQL Generalized Search Tree (GiST) indices
- Drawback: more random I/O than LSM tree

Internal node



FIN

Any questions?